Algoritmos básicos de ordenamiento--Estudio de casos

ASIGNATURA

Estructura de datos

RESPONSABLES DEL PROYECTO:

JIMENEZ MORAN VALERIA

PROFESOR DE ASIGNATURA

Maria de Monserrath Guerrero Chavero

GRUPO:DSM1SM-25ERJ

JALPAN DE SERRA, QRO.

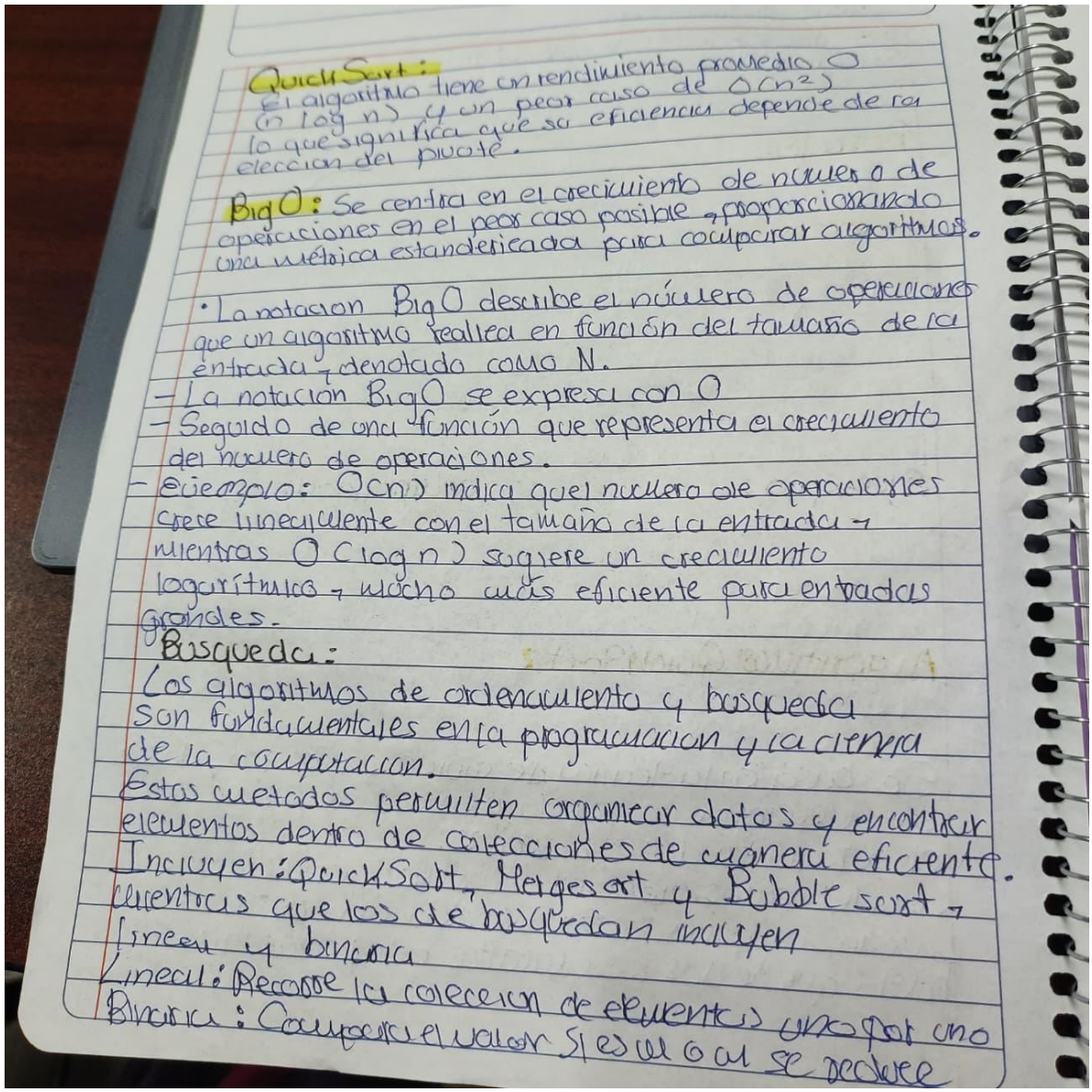
Introducción

Los algoritmos de ordenamiento y búsqueda constituyen pilares fundamentales en la estructura de datos y la ciencia de la computación, ya que permiten organizar colecciones de información y localizar elementos específicos de manera eficiente. En este trabajo se examinan algoritmos clave de ordenamiento, tales como el método de Burbuja, Inserción y QuickSort, junto con algoritmos de búsqueda como la Lineal y la Binaria. Asimismo, se analiza la notación Big O como la métrica estandarizada para evaluar la complejidad y el crecimiento del número de operaciones en el peor caso posible según el tamaño de la entrada (N).

main.py — def busqueda_binaria(arr, objetivo): Untitled-1

```python
def busqueda_binaria(arr, objetivo):
    inicio = 0
    fin = len(arr) - 1

    while inicio <= fin:
        medio = (inicio + fin) // 2
        if arr[medio] == objetivo:
            return medio
        elif arr[medio] < objetivo:
            inicio = medio + 1
        else:
            fin = medio - 1
    return -1
```

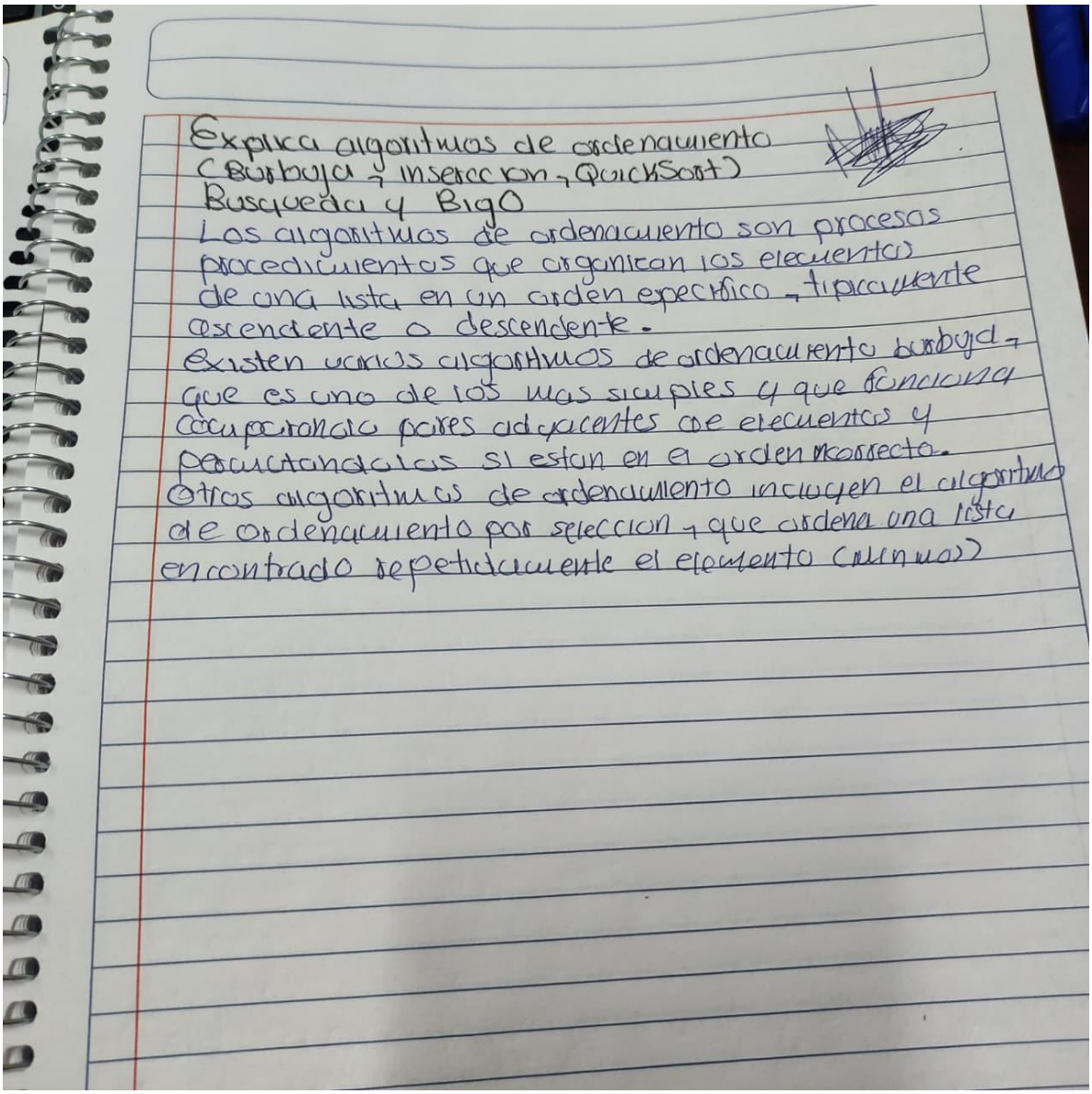


QuickSort:
El algoritmo tiene un rendimiento promedio O (n log n) y un peor caso de O(n²) lo que significa que su eficiencia depende de la elección del pivote.

BigO: Se centra en el crecimiento de número de operaciones en el peor caso posible, proporcionando una métrica estandarizada para comparar algoritmos.

- La notación BigO describe el número de operaciones que un algoritmo realiza en función del tamaño de la entrada, denotado como N.
- La notación BigO se expresa con O
- Seguido de una función que representa el crecimiento del número de operaciones.
- Ejemplo: O(n) indica que el número de operaciones crece linealmente con el tamaño de la entrada, mientras O(log n) sugiere un crecimiento logarítmico, mucho más eficiente para entradas grandes.

Busqueda:
Los algoritmos de ordenamiento y busqueda son fundamentales en la programacion y la ciencia de la computacion.
Estos metodos permiten organizar datos y encontrar elementos dentro de colecciones de manera eficiente.
Incluyen: QuickSort, Mergesort y Bubble sort, mientras que los de busqueda incluyen lineal y binaria
Lineal: Recorre la coleccion de elementos uno por uno
Binaria: Compara el valor si es mayor o menor se reduce

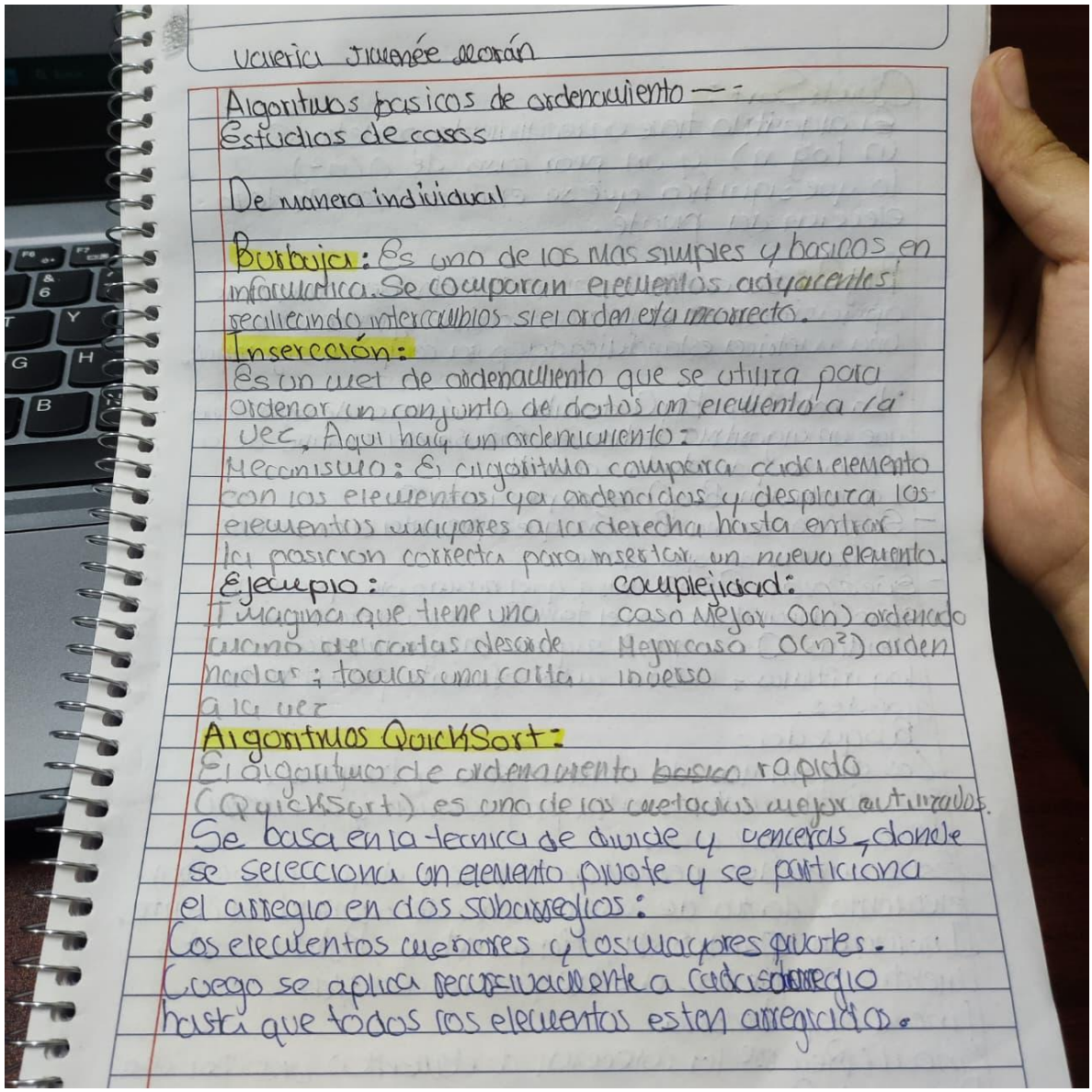


Explica algoritmos de ordenamiento (Burbuja, Insercion, QuickSort) Busqueda y BigO

Los algoritmos de ordenamiento son procesos procedimientos que organizan los elementos de una lista en un orden especifico, tipicamente ascendente o descendente.

Existen varios algoritmos de ordenamiento burbuja, que es uno de los mas simples y que funciona comparando pares adyacentes de elementos y intercambiandolos si estan en el orden incorrecto.

Otros algoritmos de ordenamiento incluyen el algoritmo de ordenamiento por seleccion, que ordena una lista encontrado repetidamente el elemento (minimo)



Valeria Jiménez Morán

Algoritmos básicos de ordenamiento —:
Estudios de casos

De manera individual

Burbuja: Es uno de los más simples y básicos en informática. Se comparan elementos adyacentes, realizando intercambios si el orden era incorrecto.

Inserción:
Es un met de ordenamiento que se utiliza para ordenar un conjunto de datos un elemento a la vez. Aquí hay un ordenamiento:
Mecanismo: El algoritmo comparará cada elemento con los elementos ya ordenados y desplaza los elementos mayores a la derecha hasta encontrar la posición correcta para insertar un nuevo elemento.
Ejemplo: Imagina que tiene una mano de cartas desordenadas; tomas una carta a la vez
complejidad:
caso Mejor O(n) ordenado
Peor caso $O(n^2)$ orden inverso

Algoritmos QuickSort:
El algoritmo de ordenamiento básico rápido (QuickSort) es uno de los métodos mejor optimizados. Se basa en la técnica de divide y vencerás, donde se selecciona un elemento pivote y se particiona el arreglo en dos subarreglos:
Los elementos menores y los mayores pivotes.
Luego se aplica recursivamente a cada subarreglo hasta que todos los elementos estén arreglados.

Conclusión

La selección adecuada de un algoritmo depende directamente del volumen de datos a procesar y de los requerimientos de eficiencia de la aplicación. Aunque métodos más simples como el ordenamiento por Burbuja o la búsqueda Lineal ofrecen un funcionamiento básico intuitivo, resultan poco eficientes ante entradas voluminosas. En contraste, algoritmos como QuickSort o la Búsqueda Binaria demuestran un rendimiento superior al emplear técnicas como "divide y vencerás" y la reducción sistemática del rango de comparación. El análisis mediante la notación Big O permite determinar objetivamente la solución técnica óptima para garantizar el correcto rendimiento del software.